API3 DAO migration proposal

Audit report

2026-10-28

Subject	The vote script, scripts and tests of `api3dao/api3-dao-migration-proposal`, and the API3 DAO contracts on Ethereum mainnet that the vote acts on
Audited revision	`a920489e0e0b0eab1abb2f9858623f3c1ffe419e`, `main` after pull request 12 (2026-10-28). Version 1.1 re-reviewed the two files pull request 15 changed, `VerifyVote.s.sol` and the README, at `4740c29551fc8be8bd21ecab9d5baeec8d8460d2`. Version 0.1 reviewed `0e10333`
Chain state	Ethereum mainnet at block 26,287,968 (2026-10-28), the vote bytes rebuilt at that block as the README names it. The event scans of version 0.1 ran to block 26,275,766 and were extended to 26,287,968
Prepared by	API3 Leadership, using Claude (Anthropic, model Fable 5.1) as the review tool
Version	1.1, see the history below

Version	Date	Change
0.1	2026-10-26	The initial review of commit `0e10333`
0.2	2026-10-27	Statuses after Leadership's review: F-1 and F-2 open with their intended resolutions, everything else acknowledged. No change to the findings
1	2026-10-28	The review redone on `a920489` (pull request 12), which drops the PNK and wPOKT calls after secondary votes 133 and 134 took them over: 26 calls, the two secondaries decoded and rehearsed with the primary in both orders, F-2 fixed for those two tokens, F-12 added, every number re-measured
1.1	2026-10-28	F-1 fixed in `4740c29` (pull request 15), which changes `VerifyVote.s.sol` and the README and no bytes, rehearsed on a fork. A-3 and the burner line of the posting checklist corrected to the decision of 2026-10-26

What this report is. It stands in for the report an auditing firm would write for this vote. API3 Leadership produced it with an AI model as the review tool: the model read the sources, ran the chain reads and the event scans, wrote and ran the fork rehearsals and drafted the text, and Leadership directed the review. Every claim is backed by a source read, a chain read at a named block, a fork test or an independent decoding, and the method is described so that any holder can redo it. It is not an independent third-party audit. Nothing in it was broadcast to mainnet.

1. Summary

The vote does what the proposal says and, once it executes, no vote can be created on the Aragon DAO again. What survives is the authority of a vote created before execution and passed afterwards, accepted under F-3 and R-1, and no vote that exists today can act (F-11) apart from the two secondaries posted for this migration. The 26 calls were decoded independently of the repository's decoder from bytes rebuilt at the README's block and match the README's table call for call, address for address. The two secondary votes

that move the PNK and the wPOKT out of the secondary Agent, 133 and 134, were read from the chain, carry the whole balances to the wei, and were rehearsed with the primary in both orders. Every permission a call relies on exists on chain today, every permission the DAO ever created was enumerated from the ACL's own events and accounted for after execution, everything the two Agents own was enumerated and either moves or is listed as left behind, and the repository's fork suite passes at the latest block. No path was found by which the vote fails to reach its end state, loses funds, or leaves a route to new votes.

Findings: no critical, high or medium issue. Four low findings, all operational or optional, and eight informational notes. F-12 is new in this version and procedural: both secondary votes have to pass, since one that fails after the primary has executed is final, so the primary is posted after both have executed or once both carry the support to pass. As of version 1.1, F-2 is fixed in the audited revision for the PNK and the wPOKT, F-1 is fixed in `4740c29`, and everything else is acknowledged by Leadership. One timing fact to plan around: the bytes cannot be built once the supply reaches 200,000,000, which at the current reward rate happens with the mint of 2027-01-14. The residual risks the project accepted before this review are restated with the evidence behind them, so that the reader sees them in one place.

Severity	Count	Ids
Critical	0	
High	0	
Medium	0	
Low	4	F-1, F-2, F-3, F-12
Informational	8	F-4 to F-11

The four low findings in one line each:

- **F-1** The byte for byte check in `VerifyVote.s.sol` fails after any third party burns one wei of API3 or sends one wei of USDC or ETH to the secondary Agent between the build and the posting, and the Vault cannot repost for seven days. Fixed: a second form takes the build block and names what moved.
- **F-2** Execution depended on three token contracts the DAO does not control. Fixed for two of them: the PNK and wPOKT calls are gone and those tokens leave through the secondaries, where a stall is recoverable. A USDC blacklisting or pause by Circle still reverts the primary until lifted, acknowledged.
- **F-3** A vote created before execution and passed afterwards can still reach the staking pool's setters through the primary Agent. This is an accepted residual. One extra call closes the pool's part of it for 6 to 8 percent more gas, measured, and is stated for Leadership to decide on.
- **F-12** A secondary vote that closes without its quorum after the primary has executed strands its token at the secondary Agent for good, since no vote can be created again. Acknowledged: the primary is posted after both have executed, or once both carry the support to pass. In which order the three execute does not matter.

2. Scope and method

In scope

- `src/MigrationScript.sol`, the single source of the vote bytes and the metadata, and `src/Interfaces.sol`.
- `script/CreateVote.s.sol` and `script/VerifyVote.s.sol`.

- `scripts/decode-script.mjs`, `scripts/update-readme.mjs`, `scripts/pick-voters.sh`, `test.sh`, `setup.sh`, `foundry.toml`, the CI workflow and the README.
- The fork tests under `test/` and the voter set in `test/data/voters.json`.
- The API3 DAO as deployed on mainnet: the Kernel, the ACL, the EVMScriptRegistry, the two Api3Voting apps, the two Agent apps, the Api3Pool and the Api3Token, and the three Safes the vote names.

Out of scope

- The proposal text pinned on IPFS. Only its title and link are in the repository. The match between the text's call list and the script is an assumption the reader has to check against the README's table (section 8).
- The Snapshot space, its settings, the voting power strategies and the verification tool.
- The DAO dashboard and its release that renders this vote.
- The delegated vesting contracts.
- The Safe transactions around the vote: the posting, the burner opt-in, the overshoot burn, the Snapshot delegations.
- The Aragon and API3 DAO contracts themselves as code. They were read to establish what the vote relies on, not audited.

What was done

Step	How	Result
Source review	`src/`, `script/`, `scripts/`, `test/`, README, CI read in full	Sections 4, 6 and 9
Independent decoding	The bytes rebuilt with `CreateVote.s.sol` at block 26,287,968, walked by a decoder written for this review (cast for the hashes, Python for the walk), compared with the README's table	26 of 26 rows match, hash `0xd5b3d6e60f527d3cc660f00a2b4e90779eb1d69b790c60e992b452b34ef924ae`
The secondary votes	133 and 134 read with `getVote` and their StartVote logs at block 26,287,968, their bytes rebuilt from the balances and compared, the Vault's power read at their snapshot blocks	Section 4, F-2, F-12
DAO sources	`api3dao/api3-dao` at `b516800` (Api3Voting, Api3Pool, Api3Token), `aragon/aragonOS` v4.4.0 (ACL, Kernel, CallsScript, EVMScriptRunner), `aragon/aragon-apps` (Agent, Vault)	Section 3
Permission enumeration	Every ACL event since the DAO's deployment block 12,786,537 (51 events, all in that block), every Kernel and registry event, decoded, and none between blocks 26,275,766 and 26,287,968	Appendix A, no permission the script does not account for

Step	How	Result
Ownership and roles	Chain-wide scan of `OwnershipTransferred` and `RoleGranted` events naming either Agent, deployment to block 26,275,766	The token is the only contract either Agent owns, no roles
Holdings	Chain-wide scan of ERC20, ERC721 and ERC1155 transfers to either Agent, then balances at block 26,275,766, no transfer to either Agent up to block 26,287,968	Section 4, finding F-5
Chain state	Pool, token, voting app, registry and Safe parameters read with `cast` at block 26,287,968	Appendix A
Repository suite	`./test.sh` at the latest block on 2026-10-28	28 tests pass, 7 suites
Fork rehearsals	23 fork tests written for this review on top of the repository's fixture, rerun on the audited revision. They are review material, not part of the repository, and section 9 lists what the repository should add	Section 6, every finding with a rehearsal names it
Gas	`newVote` and `executeVote` measured on the fork	Appendix D

What was not done

- No live rehearsal of the Vault Safe's posting transaction. The fork tests post from an account with enough voting power, not through the Safe.
- No review of the Aragon contracts' own security. They have been live since 2018 to 2021 and the vote relies on their documented behaviour, which was read in the source.
- No check of the proposal text, see Out of scope.
- Log scans find standard ownership and role patterns. A contract that names an Agent as its admin through a custom pattern without an event would not be found. None is known.

Severity

Severity	Meaning
Critical	Loss of funds or capture of governance under realistic conditions
High	Loss or capture needing unlikely conditions, or the vote failing to reach its stated end state
Medium	A degraded end state or a stall that needs a workaround from the project
Low	Operational friction, a griefable check, a dependence on a third party, or a loss the project's own procedure rules out

Severity	Meaning
Informational	A note on tests, documentation or a consequence the reader should know

Status is one of Open, Fixed in a named commit, or Acknowledged, which means the project accepts the finding as it stands. Closed marks a check that found nothing to do.

3. The system

The Aragon DAO

The API3 DAO is an Aragon DAO deployed in block 12,786,537 (2021-07-08) from a custom template. It has a Kernel, an ACL, an EVMScriptRegistry with one executor (CallsScript), two Api3Voting apps and two Agent apps. Voting power comes from the Api3Pool, not from a MiniMe token: a vote's `votingPower` is the pool's total shares at the snapshot block and a voter's weight is their shares plus shares delegated to them. Every app except the registry, which is a pinned proxy, is an upgradeable proxy whose code the Kernel's `APP_MANAGER_ROLE` can swap.

A primary vote passes when yea exceeds half of all shares, at any time while it is open, or at close when yea exceeds half of yea plus nay and half of all shares. Both branches need an absolute majority of all shares, and the share count includes shares of users mid-unstake, who hold no voting power, so the bar is slightly above half of the power that can vote. A secondary vote needs more than half of the votes cast and more than 15 percent of all shares. A vote never expires: once passed it is executable by anyone, forever. `executeVote` has no access check. The voting app runs the script through CallsScript with an empty blacklist, so a script may call any contract, the pool included.

App	Address	Role in the vote
Kernel	`0x593EA926ee9820A933488B6A288433C387D06dba`	Holds the app registry, its recovery vault is the primary Agent
ACL	`0x1E7eCC6D3B5b4cfDfC71CB7C3eA9AC4A55F4195A`	Every permission below
EVMScriptRegistry	`0x3d9AA440ECbbf7927200610CDd79A5a7C2388aaD`	Executor 1 is CallsScript `0x5cEb19e1890f677c3676d5ecDF7c501eBA01A054`
Primary voting app	`0xDb6C812E439ce5c740570578681ea7aaDba5170b`	Executes the script, 50 percent support and quorum
Secondary voting app	`0x1c8058E72E4902B3431Ef057E8d9a58A73F26372`	50 percent support, 15 percent quorum
Primary Agent	`0xD9F80Bdb37E6Bad114D747E60cE6d2aaF26704Ae`	Owns the token, is the pool's `agentAppPrimary`, holds 6,083,873.53 API3
Secondary Agent	`0x556ECbb0311D350491Ba0EC7E019c354D7723CE0`	Holds the USDC and ETH the vote moves, and the PNK and wPOKT that secondary votes 133 and 134 move

The permissions, as they stand

All 16 grants and all 16 managers were created in the deployment block and never changed. Decoded from the ACL's events, see Appendix A for the raw table.

App	Role	Holder	Manager
ACL	CREATE_PERMISSIONS_ROLE	primary voting app	primary voting app
Kernel	APP_MANAGER_ROLE	primary voting app	primary voting app
EVMScriptRegistry	REGISTRY_ADD_EXECUTOR_ROLE, REGISTRY_MANAGER_ROLE	primary voting app	primary voting app
Primary Agent	EXECUTE_ROLE, RUN_SCRIPT_ROLE, TRANSFER_ROLE	primary voting app	primary voting app
Secondary Agent	EXECUTE_ROLE, RUN_SCRIPT_ROLE, TRANSFER_ROLE	secondary voting app	primary voting app
Both voting apps	CREATE_VOTES_ROLE	any address	primary voting app
Both voting apps	MODIFY_SUPPORT_ROLE, MODIFY_QUORUM_ROLE	primary Agent	primary voting app

No permission carries parameters, and no address other than the any-address entity was ever granted CREATE_VOTES_ROLE. This matters for calls 11 and 12: revoking the any-address grant leaves nobody who can create a vote.

The token and the pool

Api3Token is an OpenZeppelin ERC20 with an owner that can grant and revoke minter status, a self-service burner status, and `renounceOwnership` overridden to revert. The primary Agent has owned it since block 21,063,473. The pool is the only address ever given minter status. Supply at block 26,287,968 is 189,123,123.33 API3, growing by one reward mint per weekly epoch (about 951,000 API3 at the current 69 percent APR on 71.6 million staked).

The pool mints its reward in `mintReward()`, which anyone can call and which every staking action calls. It mints only if the pool has minter status and always advances the epoch cursor, so once the status is gone the reward logic is inert and the rest of the pool is untouched. The pool's setters are gated on the Agent addresses it stores (`agentAppPrimary` for claims managers, the unstake wait period, the APR step and the proposal threshold, either Agent for the stake target and the APR bounds) and `setDaoApps` on the primary Agent. No claims manager was ever set and no claim was ever paid.

The Safes

Safe	Address	Version	Owners	Threshold	Modules, guard
Vault Safe	`0x8e03609ed680B237C7b6f8020472CA0687308b24`	1.3.0	five	3	none
Agent Safe	`0xBBE0dE9757F93e3306aDBfeBE906AB285EDD13DA`	1.4.1	the same five	3	none

Safe	Address	Version	Owners	Threshold	Modules, guard
Treasury Safe	`0xb7fa2Cb302C5dC127e61F27b92beC411cbf0AE4A`	1.5.0	the same five	3	none

All three use the canonical singleton and fallback handler of their version, checked against `safe-global/safe-deployments`. The Vault Safe posts the vote: it holds 3,371,901 pool shares by delegation, 21.47 percent of all shares and 214 times the 0.1 percent proposal threshold, and its last proposal was on 2026-06-16, so the seven day proposer cooldown is long over. It held the same 21.47 percent at the snapshot blocks of secondary votes 133 and 134, enough for their 15 percent quorum on its own. The Treasury Safe is not opted in as a burner, by decision.

4. What the vote does

One CallsScript of 26 calls, 4,316 bytes. Calls on the ACL and on the primary Agent are made by the primary voting app itself, everything else goes through the primary Agent's `execute`. The amounts are the balances at the block the bytes are built at. The table below gives, for each group, what it relies on and what it leaves behind. The addresses are in the README's table and were checked against the bytes. Version 0.1 audited 28 calls: calls 4 and 5 of that script, the PNK and wPOKT transfers, are gone and the rest is unchanged, renumbered.

Calls	What	Relies on	Leaves
1, 2	ACL grants the primary Agent TRANSFER_ROLE and EXECUTE_ROLE on the secondary Agent	The primary voting app manages both roles on the secondary Agent	Two new grants that stay after the vote (F-4)
3	The secondary Agent transfers 59,419.713966 USDC to the Agent Safe	Call 1. The Vault refuses a zero amount, so the build refuses a zero balance	The secondary Agent at zero USDC
4	The secondary Agent sends 0.057118793443618210 ETH to the Agent Safe through `execute` with empty calldata	Call 2. `transfer` would use `send` and the 2,300 gas stipend does not reach a Safe	The secondary Agent at zero ETH
5	The primary Agent transfers 6,083,873.527701856751553400 API3 to the Treasury Safe	The primary voting app holds TRANSFER_ROLE on the primary Agent	The primary Agent at zero API3
6	Token: the pool loses minter status	The primary Agent owns the token	No more reward mints
7 to 9	Token: the primary Agent becomes a minter, mints 10,876,876.674326757910422715 API3 to the Treasury Safe, stops being a minter	The mint is 200,000,000 minus the total supply at the build block	Total supply 200,000,000 if executed in the epoch it was built in
10	Token: ownership to the dead address	`renounceOwnership` reverts, this is the only way	No minter can ever be added

Calls	What	Relies on	Leaves
11, 12	ACL revokes CREATE_VOTES_ROLE from any address on both voting apps	The primary voting app manages the role	No vote can be created
13, 14	ACL removes the manager of CREATE_VOTES_ROLE on both apps	Same	The role can never be granted again
15 to 18	Both voting apps set support then quorum to one wei under 100 percent, through the primary Agent	The primary Agent holds the two modify roles on both apps. Support first, the app rejects a quorum above the support	A vote created through an unknown route would need every share
19 to 22	ACL removes the managers of the two modify roles on both apps	The primary voting app manages them	No new vote can change the settings. The primary Agent keeps the two grants, so a vote created before execution still could (F-3)
23, 24	ACL revokes the primary voting app's APP_MANAGER_ROLE on the Kernel and removes the manager	The primary voting app holds and manages it	No app's code can be swapped, ever
25, 26	ACL revokes the primary voting app's CREATE_PERMISSIONS_ROLE on the ACL and removes the manager, last	Same	No permission can be created, so calls 13, 14 and 19 to 22 cannot be undone

Everything the script relies on holds on chain at block 26,287,968 (Appendix A). The order is right where it matters: the grants before the transfers, support before quorum, the revokes before the manager removals, the ACL's own role last.

The two secondary votes

Version 0.1 found that a wPOKT pause or a PNK transfer switch would stall the whole primary (F-2). Leadership took those two tokens out of it: two secondary votes were posted through the DAO dashboard on 2026-10-27, and the audited revision drops the PNK and wPOKT calls from the primary. Read at block 26,287,968:

Vote	Created	Closes	Script	In favour today
Secondary 133	2026-10-27 18:18:47 UTC by `0x8566E1751836b32B56f9a88053Bb8cb404f86671`, snapshot block 26,285,794	2026-11-03 18:18:47 UTC	The secondary Agent's `execute` on PNK: `transfer(Agent Safe, 1,547,987 PNK)`	259,876 shares, 1.65 percent, none against

Vote	Created	Closes	Script	In favour today
Secondary 134	2026-10-27 18:51:59 UTC by `0xe0625a7B539090130f6eDCdef208f6be129a6291`, snapshot block 26,285,960	2026-11-03 18:51:59 UTC	The same on wPOKT: `transfer(Agent Safe, 249,999.99 wPOKT)`	259,876 shares, 1.65 percent, none against

Each is one CallsScript entry, the secondary Agent's `execute` on the token with an ERC20 `transfer` of the whole balance to the Agent Safe, 256 bytes. Each was rebuilt for this review from the balance at the block and compared with what `getVote` returns: identical. The metadata is the dashboard's version 2 scheme with the titles "Consolidate PNK holdings into USDC" and "Consolidate wPOKT holdings into USDC" and a link to a text. A secondary passes with more than 15 percent of all shares in favour and more yea than nay at its close, or more than half of all shares at any time. The Vault Safe held 3,371,901 shares at both snapshot blocks, 21.47 percent, enough for the quorum on its own and short of an absolute majority, so with its vote and yea above nay both execute at their close, and someone has to call `executeVote`, unless yea passes half of all shares earlier and the crossing vote executes them.

The primary neither needs them to execute first nor stops them. It revokes vote creation on the secondary app (calls 11 and 12) and raises the app's support and quorum for future votes (calls 17 and 18), but a vote keeps the thresholds it was created with, the secondary voting app keeps EXECUTE_ROLE on the secondary Agent, and `executeVote` has no access check. Both orders were rehearsed on a fork and reach the same end state, the secondary Agent empty of all four assets. What the primary does change is that once it has executed nothing can be posted again, so a secondary that fails is final (F-12).

What the DAO controls and what happens to each

This is the completeness check. Each row is something the two voting apps could reach before the vote, with its state after.

Capability	Before	After the vote
Mint API3	The pool, by minter status	Gone. The owner is the dead address, the minter set is empty
Change the token's minters	The primary Agent as owner	Gone
The primary Agent's API3	6,083,873.53	Moved to the Treasury Safe
The secondary Agent's USDC and ETH	59,419.71 USDC, 0.0571 ETH	Moved to the Agent Safe
The secondary Agent's PNK and wPOKT	1,547,987 PNK, 249,999.99 wPOKT	Moved to the Agent Safe by secondary votes 133 and 134, which have to pass (F-12). The primary no longer touches them
Other tokens at the Agents	35 other ERC20 and 7 ERC1155 holdings, mostly airdrop spam, and 1 HEX at the secondary voting app	Stay where they are for good (F-5)
The pool's setters: claims managers, unstake wait period, APR bounds and step, stake target, proposal threshold, the DAO app addresses	The Agents, through a vote	Unreachable, since no vote can be created. Reachable by a vote created before execution (F-3)

Capability	Before	After the vote
Create a vote	Any address with 0.1 percent of shares	Nobody
Change support and quorum	The primary Agent, through a vote	No new vote. The values are at the maximum and the primary Agent keeps the grants, so a vote created before execution could lower them (F-3)
Swap an app’s code, set the recovery vault	The primary voting app on the Kernel	Nobody
Create a permission, with a manager or born burned	The primary voting app on the ACL	Nobody
Grant, revoke or hand over the Agent and registry roles, or burn their manager	The primary voting app as manager	Still the primary voting app, which can only act through a vote created before execution (F-3, F-4)
Add or disable a script executor	The primary voting app	Same
Recover tokens sent to the Kernel, ACL, registry or voting apps	Anyone, to the primary Agent	Same, and the primary Agent moves them only through a vote created before execution (R-1)
The pool’s reward schedule and locked rewards	Weekly mint, 52 epoch lock	Mints stop, locks unwind on the same schedule (tested)
Staking, unstaking, withdrawing	Unchanged	Unchanged (tested)

Nothing outside the DAO depends on the Agents: the chain-wide scans found no contract owned by either Agent other than the token, no AccessControl role, no ERC721. The TimelockManager that holds early vesting is owned by a separate 2 of 3 Safe. The pool’s claims manager slot was never used.

5. Verification results

Property	How	Result
The bytes are one CallsScript of 26 calls, every call on the ACL or the primary Agent	Independent decoder, `test/vote/script.tree`	Holds
The README’s table is what the source builds at the named block	Independent decoder against the README	26 of 26 rows match
Every permission a call relies on exists	ACL events, `cast` at block 26,287,968, `_assertPermissionsTheScriptReliesOn`	Holds, with gaps in what the fixture pins (F-6)
No other grantee of CREATE_VOTES_ROLE exists	ACL events since deployment	Only the any-address entity
No permission has parameters	ACL events	None
The supply lands on exactly 200,000,000	`test/vote/execute.tree`, both paths	Holds, the late path after the Treasury Safe burns the overshoot

Property	How	Result
A burn before execution lands below the target and nothing can mint it back	`test/vote/execute.tree`	Holds, accepted (R-2)
No vote can be created on either app afterwards, even by the primary voting app	`test/dao/aragon.tree`	Holds
The root permissions are gone and unmanaged	`test/dao/aragon.tree`	Holds
The pool mints nothing and keeps its cursor	`test/pool/rewards.tree`	Holds
Stakers keep every right but voting	`test/pool/staking.tree`	Holds, four scenarios
The Agents hold nothing else of value	Holdings scan	Holds (F-5)
Nothing else is owned by the Agents	Ownership and role scans	Holds
The three Safes are canonical, same owners, no modules or guard	`cast` against safe-deployments	Holds
The Vault can post	Shares and cooldown at block 26,287,968	Holds, 214 times the threshold
The vote fits a block	Gas on the fork	Posting 3.00M, execution 1.55M
A dust deposit after posting does not block execution	Rehearsal for this review	Holds, the dust stays
A third party can stall execution through a moved token	Rehearsal for this review	Circle only, since the PNK and wPOKT calls are gone. A wPOKT pause and a PNK switch leave the primary untouched and stall only the secondary that moves that token (F-2)
A third party can break the byte for byte verification	Rehearsal for this review	Yes (F-1)
No existing vote on either app can change anything, before or after the migration	`canExecute` on all 150 votes at block 26,283,321, the six that pass it executed on a fork before and after	Holds, five revert for good, one moves nothing (F-11)
Secondary votes 133 and 134 are exactly a transfer of the whole PNK and wPOKT balances to the Agent Safe	Rebuilt from the balances at block 26,287,968 and compared with `getVote`	Holds, byte for byte
The secondaries pass with the Vault’s vote and no vote against, and execute before or after the primary	Rehearsal for this review, both orders	Holds, the Vault’s 21.47 percent clears the 15 percent quorum and the secondary Agent ends empty of all four assets
A secondary that closes without its quorum after the primary executed is final	Rehearsal for this review	Yes, the token is stranded (F-12)

6. Findings

F-1 One wei from anyone makes the posted vote fail verification, and the Vault cannot repost for a week

Severity: Low. Status: Fixed in `4740c29` on 2026-10-28, as decided on 2026-10-27. It changes `VerifyVote.s.sol` and the README and no bytes.

`VerifyVote.s.sol` rebuilds the script from the source at the vote's snapshot block and requires the posted bytes to be identical. The bytes carry four amounts read at the build block: the mint, the primary Agent's API3, the secondary Agent's USDC and its ETH. Between the build and the posting transaction, which a Safe signs over minutes to hours, any of the following moves a balance and changes the bytes:

- Any holder opts in as a burner and burns one wei of API3. 21 addresses are opted in today and 100.28 API3 were burned in block 26,234,257 (2026-10-20) by one of them. The mint amount is the only one that depends on the supply.
- Anyone sends one wei of USDC or one wei of ETH to the secondary Agent.

The vote itself is unaffected, it executes with the amounts it carries and leaves the dust behind, and the dashboard's allowlist is keyed on the hash of the bytes that were posted, so it is unaffected too. Before the fix, `VerifyVote` ended with a bare revert instead of `VERIFIED`, which is what the README told holders to expect. If the project decides to repost, the Vault hits `API3_HIT_PROPOSAL_COOLDOWN` for seven days from the first posting, a vote posted by another address fails `VerifyVote`'s creator check, and the reposted bytes carry new amounts and a new hash, so the dashboard has to be released again.

Rehearsed on a mainnet fork for this review: the bytes were built, the Vault opted in as a burner and burned one wei, and the bytes rebuilt in the next block differ by hash. The same with one wei of USDC sent to the secondary Agent. A second `newVote` from the Vault right after a first one reverts with `API3_HIT_PROPOSAL_COOLDOWN` and succeeds seven days and two seconds later.

```
bytes memory built = buildScript(readAmounts());
vm.startPrank(VAULT);
API3.updateBurnerStatus(true);
API3.burn(1);
vm.stopPrank();
vm.roll(block.number + 1);
assertNotEq(keccak256(built), keccak256(buildScript(readAmounts())));
```

Fix. The exact check stays the first form. `run(uint256 voteId, uint256 buildBlock)` rebuilds the script at the block `CreateVote.s.sol` wrote to `proposal/generated/amounts.json`, prints the four amounts at the build block next to the ones at the snapshot block with the ones that moved marked, and requires the posted bytes to be that build. The failure message of the first form names the second, and the README says what a mismatch means. Rehearsed on an anvil fork of mainnet for this version: the bytes built at block 26,288,515, one wei of USDC sent to the secondary Agent from the Agent Safe, the vote posted as the Vault in block 26,288,517, `run(17)` fails with that message, `run(17, 26288515)` reports the USDC moved up by one wei and the other three amounts unchanged, and ends VERIFIED. Any block at which the source builds the same bytes passes the second form, so the block is a witness, not a claim about when the build happened. Limits. The second form reads the four amounts at the snapshot block before it tries the build block, so the guards of `readAmounts` apply there: a zero API3 balance at the primary Agent, a zero USDC or ETH balance at the secondary Agent, or a supply at 200,000,000, stops it before the fallback. A zero balance needs a vote that drains an Agent between the build and the posting, and breaks execution as well (R-4).

The supply needs about eleven more weekly mints and breaks only the build and the verification: the token's `mint` has no cap, so posted bytes still mint their fixed amount and overshoot by the rewards minted since the build (R-5). What remains is the week of cooldown and the dashboard re-release if the project chose to repost, so the build and the posting stay in one sitting.

F-2 Three token contracts outside the DAO can stall execution

Severity: Low. Status: Fixed in `a920489` for the PNK and the wPOKT, acknowledged on 2026-10-27 for the USDC.

One reverting call reverts the whole script. At version 0.1, calls 3 to 5 transferred three tokens whose contracts can refuse:

Token	Control	Who	History
wPOKT	`pause()`	A 2 of 4 Safe, `0x2f16615234827eE4dF14d02d40C24E6a258dD360`	Paused from 2025-06-03 to 2025-06-09
PNK	`enableTransfers(false)` on the MiniMe token	The controller, KlerosLiquid `0x988b3A538b618C7A603e1c11Ab82Cd16dbE28069`. Whether its governance exposes the call was not read for this review	Transfers enabled today
USDC	Blacklisting of the sender or the recipient, or `pause()` on the whole token	Circle	Neither the secondary Agent nor the Agent Safe is blacklisted, the token is not paused

What changed. Secondary votes 133 and 134, posted on 2026-10-27, move the PNK and the wPOKT to the Agent Safe (section 4), and the audited revision drops the two calls from the primary, which now moves the USDC and the ETH. A wPOKT pause or a PNK transfer switch stalls only the secondary that moves that token, which stays decided and executes once the control is lifted, before or after the primary. What remains in the primary is Circle. While a blacklisting of the secondary Agent or the Agent Safe, or a pause of the token, is active, `executeVote` reverts. The vote stays decided and executes once the control is lifted, and a vote never expires, so nothing is lost. Two consequences for the procedure: a voter whose vote crosses the majority with `executesIfDecided` set, which is how the dashboard votes, has the whole transaction revert and has to vote with the flag off through another tool. And a stall that outlasts the project's patience cannot be worked around without a second vote that drops the stuck token, which needs the same majority again.

Rehearsed on a mainnet fork for version 0.1, once per token: the vote is passed, the control is applied by its holder (the pauser Safe, the PNK controller, Circle's blacklister), `executeVote` reverts, the control is lifted, and the vote executes to the full end state. Rehearsed again for version 1 on the audited revision: with wPOKT paused or PNK transfers disabled the primary executes to its end state and leaves the token for its secondary, and a paused wPOKT stalls vote 134 at its close until the pauser unpauses, after which it executes. The USDC rehearsal still stalls the primary.

```
vm.prank(WPOKT_PAUSER);
IPausable(address(WPOKT)).pause();
executeNow();
_assertOutcome();
assertEq(WPOKT.balanceOf(address(AGENT_SECONDARY)), 249_999_990000, "left for secondary
vote 134");
```

Recommendation. Done for the PNK and the wPOKT. For the USDC, which stays in the primary with Circle as the one third party: check `isBlacklisted` for the secondary Agent and the Agent Safe and `paused()` right before posting (section 10), and write the fallback into the signers' and voters' notes: if a stall appears, vote with `executesIfDecided` off and call `executeVote` once it clears.

F-3 A vote created before execution keeps a route to the staking pool, and one call would close it

Severity: Low. Status: Acknowledged on 2026-10-27, the option below not adopted. An unwanted vote is handled operationally, and more on-chain calls are a bad trade for a case that needs the migration's own majority.

A primary vote created before the migration executes keeps the support and quorum it was created with and never expires, and the primary voting app keeps every role it holds on the primary Agent. So a vote created before execution and passed afterwards, which needs the same absolute majority as the migration itself, can still drive the primary Agent. After the migration and the two secondaries the Agents hold nothing of value, so the one place such a vote can still do harm is the pool, through `agentAppPrimary`: set a claims manager and pay out staked API3 (71.6 million today), set an unstake wait period with no upper bound, or point `setDaoApps` elsewhere. The project accepted this on 2026-10-21 after measuring a full role cleanup at 22 extra ACL calls that more than doubled the gas, and it keeps the operational rule that no vote other than secondaries 133 and 134 is open when the migration is posted or while it runs.

A single extra call closes the pool's part without touching the roles: the primary Agent calls `setDaoApps(dead, dead, primary voting app, secondary voting app)`. The pool then has no Agent, the voting app slots stay as they are so that the dashboard's proposal history, which reads them through the Convenience contract, keeps working, and stakers are unaffected. Measured on the fork with the call appended as call 27:

	26 calls	**27 calls**	**Difference**
Script bytes	4,316	4,632	+316
`newVote` gas	2,995,478	3,181,913	+186,435
`executeVote` gas	1,549,869	1,680,063	+130,194

```
// one more entry: the primary Agent's execute on the pool, no value, with the calldata
abi.encodeCall(IApi3Pool.setDaoApps, (DEAD, DEAD, address(VOTING),
address(VOTING_SECONDARY)))
```

What the call does not close: the Agent and registry roles stay with the voting apps, so a stale vote can still move whatever lands at the Agents later and run code in an Agent's context through a registry executor. The dashboard's treasury display reads the Agents' balances through the pool's slots and would show the dead address's balance, which the planned post-execution dashboard change can take into account.

Rehearsed on a mainnet fork for this review: the 27 call script executes to the full end state, every pool setter refuses the primary Agent afterwards with `Pool: Caller not primary agent`, and a staker unstakes and withdraws as before.

Recommendation. The numbers above are what it costs, and the closure is of the one capability a stale vote could use against stakers rather than against the empty Agents. Adopting it would change the bytes and the proposal text's call list. Not adopted.

F-4 Calls 1 and 2 add two permanent grants

Severity: Informational. Status: Acknowledged on 2026-10-27.

After the vote, the primary Agent holds TRANSFER_ROLE and EXECUTE_ROLE on the secondary Agent, managed by the primary voting app. They are inert for the same reason every remaining Agent role is inert, nothing can create a vote, and they widen what a vote created before execution can reach by the secondary Agent's leftovers, which include the PNK and the wPOKT between the primary's execution and the secondaries' (F-12). Revoking them at the end would be two ACL calls. Checked on the fork after execution.

F-5 What the vote leaves at the Agents

Severity: Informational. Status: Acknowledged on 2026-10-27, nothing of value is left behind.

The holdings scan over every transfer ever made to the Agents, with balances at block 26,275,766 and no transfer to either Agent up to block 26,287,968:

- Secondary Agent, beside the USDC and ETH the vote moves and the PNK and wPOKT the secondaries move: WETH 0.0000001, GRG 0.025, DeSci 899.325, DECT 921, STEAK 124.271421, MTV 1000, yRise 0.000001, YFTE 0.01, 27 airdrop or scam tokens, and five ERC1155 holdings (Unisocks NFT, Nft Mint Club Official and three spam contracts).
- Primary Agent: two ERC1155 holdings, "API3 Mysterybox NFT" (`0x5be2aa50252802e399fdf40747e17430ad8b0c26`) and a spam contract.
- Secondary voting app: 1 HEX. Anyone can move it to the primary Agent with `transferToVault`, where it stays.

None of this can move once the vote executes, other than through a vote created before execution (R-1). The scan found no ERC721 at either Agent and no ETH at the primary Agent.

F-6 The fixture does not pin every permission the script relies on

Severity: Informational. Status: Acknowledged on 2026-10-27.

`_assertPermissionsTheScriptReliesOn` in `test/Base.t.sol` checks nine facts. The script also relies on the primary voting app managing EXECUTE_ROLE on the secondary Agent (call 2), on the primary Agent holding MODIFY_QUORUM_ROLE on the primary app and MODIFY_SUPPORT_ROLE on the secondary app (calls 16 and 17), on the primary voting app holding TRANSFER_ROLE and EXECUTE_ROLE on the primary Agent (calls 5 to 10 and 15 to 18), and on it managing CREATE_VOTES_ROLE and the two modify roles on both apps (calls 13, 14 and 19 to 22). The fork execution covers all of them, but a change on chain would surface as a revert inside the script rather than as a named precondition. The missing assertions:

```
assertEq(ACL.getPermissionManager(address(AGENT_SECONDARY), EXECUTE_ROLE),
address(VOTING));
assertTrue(ACL.hasPermission(address(AGENT), address(VOTING), MODIFY_QUORUM_ROLE));
assertTrue(ACL.hasPermission(address(AGENT), address(VOTING_SECONDARY),
MODIFY_SUPPORT_ROLE));
assertTrue(ACL.hasPermission(address(VOTING), address(AGENT), TRANSFER_ROLE));
assertTrue(ACL.hasPermission(address(VOTING), address(AGENT), EXECUTE_ROLE));
assertEq(ACL.getPermissionManager(address(VOTING), CREATE_VOTES_ROLE), address(VOTING));
assertEq(ACL.getPermissionManager(address(VOTING_SECONDARY), CREATE_VOTES_ROLE),
address(VOTING));
assertEq(ACL.getPermissionManager(address(VOTING), MODIFY_SUPPORT_ROLE), address(VOTING));
assertEq(ACL.getPermissionManager(address(VOTING), MODIFY_QUORUM_ROLE), address(VOTING));
assertEq(ACL.getPermissionManager(address(VOTING_SECONDARY), MODIFY_SUPPORT_ROLE),
address(VOTING));
assertEq(ACL.getPermissionManager(address(VOTING_SECONDARY), MODIFY_QUORUM_ROLE),
address(VOTING));
```

F-7 Two suites rehearse the same vote

Severity: Informational. Status: Acknowledged on 2026-10-27.

`test/MigrationProposal.t.sol` is an older copy of the fixture with its own eight tests. Everything it tests is also tested by the five `.tree` suites, which are the ones the README describes. CI runs both, which doubles the fork time, and the file is the one place where the metadata is a fixed string rather than the proposal files. The project decided on 2026-10-21 to keep it, before the trees existed. Now that they do, removing it or listing it in the README would be tidier, nothing more.

F-8 The metadata test proves determinism, not content

Severity: Informational. Status: Acknowledged on 2026-10-27.

`test_ShouldCarryTheMetadataBuiltFromProposaltitletxtAndProposaldescriptionmd` rebuilds the string with the same function and the same files and compares. It passes for any content, including the `TODO: IPFS link` the description holds today. `VerifyVote.s.sol` is where the content is checked against the posted vote, at the revision the verifier has checked out. A line in the README that the check runs at the posted revision would close the gap. A nit.

F-9 The crossing voter pays for execution

Severity: Informational. Status: Acknowledged on 2026-10-27.

The dashboard votes with `executesIfDecided` set, so the vote that takes yea past half of all shares executes the 26 calls in the same transaction, about 1.55 million gas on top of the vote. Worth a line in the voter instructions, nothing more.

F-10 Consequences the holders should read before voting

Severity: Informational. Status: Acknowledged on 2026-10-27, for the proposal text.

- The token can never have a minter again. The owner is the dead address and `renounceOwnership` is not the route, `transferOwnership` is. There is no undo.

- The pool's parameters stay as they are, with no new vote able to change them: a seven day unstake wait, no claims manager, the current APR bounds and stake target, and the DAO app slots. A vote created before execution and passed afterwards could still reach them through the primary Agent (F-3), and F-3's call, if adopted, closes that route and freezes the slots at the dead address instead.
- The secondary voting app's 135 votes and the primary's 17 stay readable, and any of them that passed and was never executed stays executable (R-1). Secondary votes 133 and 134 are meant to be: if they pass, they execute at their close whether or not the primary has executed by then.
- The vote can be executed by anyone the moment yea passes half of all shares, before the seven days end.

F-11 The votes that already exist cannot change anything

Severity: Informational. Status: Closed, nothing to do.

The stale vote residual (R-1) is about votes created between now and execution. The 17 primary and 133 secondary votes that existed at block 26,283,321 (2026-10-27) were checked one by one. None was open. Four primary and twelve secondary votes failed and can never execute. Six secondary votes passed and were never executed, so `canExecute` is true for them, and each is a single USDC call through the secondary Agent:

Vote	What the script does	Executing it
10, 12, 13, 77	Calls a selector USDC does not have	Reverts with `EVMCALLS_CALL_REVERTED`, for good
42	`transfer(USDC's own address, 0)`	Reverts, Circle blacklists the token's own address
53	`transfer(0x07c79abc…, 0)`	Succeeds and moves nothing

Rehearsed on a fork for this review, all six before the migration and again after it executes: the same five revert and vote 53 executes with every balance of the secondary Agent unchanged. The migration's calls 17 and 18 do not touch them, a vote keeps the support and quorum it was created with.

Secondary votes 133 and 134 were created later that day for this migration. They are the two deliberate exceptions, decoded in section 4 and rehearsed under F-12. No other vote was created up to block 26,287,968.

F-12 A secondary that closes without its quorum after the primary has executed strands its token

Severity: Low. Status: Acknowledged on 2026-10-28.

The primary's calls 11 and 12 revoke vote creation on both apps for good. A secondary vote that exists by then keeps running: it can be voted on, it passes with more than 15 percent of all shares in favour and more yea than nay at its close, and it executes afterwards, since the secondary voting app keeps its roles on the secondary Agent (section 4). But a secondary that closes short of its quorum after the primary has executed is final. It can never execute, nothing can be posted to try again, and its token stays at the secondary Agent with no vote able to move it.

Today that is 1,547,987 PNK on vote 133 and 249,999.99 wPOKT on vote 134, each at 259,876 shares in favour, 1.65 percent of the 15,701,999.75 shares they count against. The Vault Safe held 3,371,901 shares at both snapshot blocks, 21.47 percent, so its vote alone clears the quorum. Passing still needs yea above

nay at the close. The Vault's share is not an absolute majority, so with it alone neither vote can execute before its close on 2026-11-03 at 18:18:47 and 18:51:59 UTC, and someone has to call `executeVote` on each afterwards. More yea than half of all shares would execute a secondary before its close. The primary posted on 2026-10-29 executes the moment yea crosses half of all shares, which can be before 2026-11-03. The order does not matter, the quorum and the majority do.

Rehearsed on a mainnet fork for this review on top of the repository's fixture: with the Vault's vote both secondaries close passed and execute, before the primary and after it, and the secondary Agent ends empty of all four assets. Without the Vault's vote, after the primary has executed, vote 133 closes with `canExecute` false, `executeVote` reverts with `VOTING_CAN_NOT_EXECUTE`, `newVote` on the secondary app reverts with `APP_AUTH_FAILED` for anyone, and the PNK stays.

```
executeNow();                                    // the primary
vm.warp(1793731920);                             // 2026-11-03 18:52 UTC, both closed,
nobody voted
assertFalse(VOTING_SECONDARY.canExecute(133));
vm.expectRevert(bytes("VOTING_CAN_NOT_EXECUTE"));
VOTING_SECONDARY.executeVote(133);
vm.prank(VAULT);
vm.expectRevert(bytes("APP_AUTH_FAILED"));
VOTING_SECONDARY.newVote(hex"00000001", "", true, false);
```

Recommendation. Post the primary after both secondaries have executed, which makes Thursday 2026-11-05 the first posting sitting, or confirm before posting that both carry the support to pass: the Vault's yea on each, one Safe transaction per vote, clears the quorum, and nothing votes against. The posting checklist carries the check (section 10). After 2026-11-03 18:52 UTC someone calls `executeVote(133)` and `executeVote(134)` and reads the two balances at zero.

7. Acknowledged residual risks

Decided by the project before this review and restated here with the evidence.

Id	Residual	Evidence	Decided
R-1	A vote created before execution, open or passed and unexecuted, can drive the Agents afterwards. On the primary app it needs more than half of all shares voting yea, the majority the migration itself needs, and reaches the pool through the primary Agent. On the secondary app it needs more than 15 percent of all shares voting yea and more yea than nay, and drives only the secondary Agent, which reaches no pool setter and holds nothing of value once 133, 134 and the migration have executed. Mitigation is the rule that no vote other than secondaries 133 and 134 is open when the migration is posted or while it runs. See F-3 for the pool and the option, F-11 for the votes that existed before, none of which can act, and F-12 for the two secondaries	Reproduced on a fork in the 2026-10-20 review, 22 call cleanup measured and dropped	2026-10-21
R-2	A burn before execution lands the supply below 200,000,000 with no minter to make it up	`test/vote/execute.tre e`, burns are live	2026-10-20
R-3	Tokens that arrive at either Agent after posting stay there	Rehearsed for this review with one wei of USDC	2026-10-20
R-4	A secondary vote that pays out USDC or ETH of the secondary Agent before execution makes the whole script revert until the balance is restored. Anyone can restore it. Votes 133 and 134 pay out PNK and wPOKT, which the primary no longer moves, so they do not trigger this	2026-10-20 review, crossing vote tests since dropped, the two secondaries rehearsed for version 1	2026-10-21

Id	Residual	Evidence	Decided
R-5	A reward mint between posting and execution overshoots the target by that epoch’s reward. The Treasury Safe, where the mint lands, burns the surplus in one transaction that opts in as a burner, burns and opts out (A-3)	`test/vote/execute.tree` late path	2026-10-25

8. Assumptions outside the scope

Each of these has to hold for the vote to do what the holders expect, and none can be verified from the audited repository alone. Leadership acknowledged all eight on 2026-10-27. A-1 and A-5 changed with the secondaries and A-3 was corrected to the decision of 2026-10-26, so those three are restated here for a second acknowledgement.

Id	Assumption	Where it is settled
A-1	The call list in the pinned proposal text names the same 26 calls, addresses and amounts as the README’s table at the posting block, and names the two secondaries for the PNK and the wPOKT	The proposal text
A-2	The dashboard release that renders a multi-call vote, keyed on this script’s hash, is live before the vote is posted	The dashboard repository
A-3	After a late execution the Treasury Safe burns the overshoot in one transaction that opts in as a burner, burns and opts out, decided 2026-10-26, so it is not a burner before the vote	A Treasury Safe transaction of three calls, prepared by the project as a Transaction Builder batch
A-4	The Vault signers check the `newVote` calldata against the source before signing, and the build and the posting happen in one sitting right after a reward mint	safe-tx-verifier, the posting procedure
A-5	No vote other than secondaries 133 and 134 is open on either app when the migration is posted, none is created while it runs, and both secondaries pass (F-12)	The posting procedure

Id	Assumption	Where it is settled
A-6	After execution the Vault moves its own API3 to the Treasury Safe, and the Treasury and Agent Safes delegate their Snapshot power to the dead address	Safe transactions
A-7	The Snapshot space, its strategies and the verification tool are ready before the first proposal there	Leadership, outside the audited repository
A-8	`test/data/voters.json` is regenerated before posting so the suite keeps passing the vote with current holders	`scripts/pick-voters.sh`

9. Test coverage

The five trees and their tests cover the bytes, the end state on both execution paths, the voting apps afterwards, the reward clock and four staking scenarios. They pass at the latest block in about two minutes. The rehearsals written for this review are not part of the repository. What the suite does not cover, and what the repository should add:

Gap	What to add
The permissions that are relied on but not pinned	The eleven assertions of F-6 in `_assertPermissionsTheScriptReliesOn`
Circle stalling execution	One test: Circle's blacklister blacklists the Agent Safe, `executeVote` reverts, the blacklisting is lifted, the vote executes, the end state holds (F-2)
The two secondary votes	One test that casts the Vault's vote on 133 and 134, closes them and executes them before and after the primary, and one that shows a secondary closing without its quorum after the primary is final (F-12). Needed until they execute on mainnet, after which the balances are zero and the tests retire
A balance moving between the build and the posting	A test that builds, burns one wei, rebuilds and shows the hashes differ, and one that posts the first build on the fork and runs `VerifyVote` in both forms (F-1)
A dust deposit after posting	One test: one wei to the secondary Agent after posting, execution succeeds, the wei stays (R-3)
The bar a vote has to clear	One test: a vote with the Vault's power alone cannot execute, open or closed
Gas	Log `executeVote` gas in the execute tree, so a change in the script shows up in the numbers (F-9)
The pool hardening call, if adopted	The end state plus every pool setter refusing the primary Agent afterwards (F-3)
A vote created before execution	Not in the suite since the cleanup was dropped. If R-1 stays accepted, no test is needed, the rule is procedural

Gap	What to add
The posting transaction from the Vault Safe	Not coverable on the fixture, the Safe path is rehearsed outside the repository (A-4)

10. Preconditions for posting and execution

A checklist for the day, from what this review relied on. Reads, apart from three actions: the reward mint in item 1 when it is missing, the posting in item 6 and the two executions in item 9.

There is a deadline. The bytes cannot be built once the total supply reaches 200,000,000, since the mint would be zero or negative. At the current stake and the APR stepping down one point per epoch, the supply crosses the target with the reward mint of 2027-01-14, so the last epoch in which the vote can be built as written starts on 2027-01-07. Stakers leaving pushes the date out, stakers joining pulls it in.

1. The epoch's reward has been minted: `epochIndexOfLastReward() == block.timestamp / 1 weeks` on the pool, else call `mintReward()` first.
2. No vote is open on either app other than secondaries 133 and 134: `votesLength()` is 17 on the primary app and 135 on the secondary. Both secondaries have executed, or carry the support to pass: `getVote(133)` and `getVote(134)` show yea above 15 percent of votingPower and nay below yea (F-12).
3. The one token contract left in the primary is usable: `isBlacklisted` on USDC false for the secondary Agent and the Agent Safe, `paused()` on USDC false. `paused()` on wPOKT and `transfersEnabled()` on PNK matter only to the secondaries now, where a stall is recoverable.
4. The Treasury Safe has code. It is not opted in as a burner before the vote, by decision: the overshoot fallback is one transaction after execution that opts in, burns and opts out (A-3).
5. The Vault clears the threshold and the cooldown: `userVotingPower(vault)` against `totalShares() / 1000`, and `getUser(vault)` last proposal timestamp older than seven days.
6. `forge script script/CreateVote.s.sol` and the Safe transaction signed and sent in one sitting, with the balances the bytes carry checked against the chain right before sending.
7. `forge script script/VerifyVote.s.sol --sig "run(uint256)" <id>` ends `VERIFIED` before the vote is announced. If it fails on the bytes, `--sig "run(uint256,uint256)" <id> <build block>` with the block from `proposal/generated/amounts.json` has to end `VERIFIED` and show only the amounts that moved (F-1).
8. Voters are told the crossing vote costs about 1.55 million gas, and what to do if execution is stalled (F-2).
9. After 2026-11-03 18:52 UTC: `executeVote(133)` and `executeVote(134)` on the secondary app, then `balanceOf` of the secondary Agent on PNK and on wPOKT at zero (F-12).

Appendix A: on-chain state at block 26,287,968

Permissions

Every ACL event since deployment, 51 events in block 12,786,537, nets to the 16 grants and 16 managers in the table in section 3. Three template addresses were granted and revoked within the same block and hold nothing. No `SetPermissionParams` event exists. The Kernel's 14 events install the six apps, the registry's one event enables executor 1. The ACL, the Kernel and the registry emitted nothing between blocks 26,275,766 and 26,287,968, and no token reached either Agent in that window.

Parameters

Contract	Value
Token owner	primary Agent
Token minters	the pool only
Total supply	189,123,123.325673242089577285 API3
Pool total stake	71,629,258.55 API3
Pool total shares	15,702,339.54
Pool API3 balance	73,933,013.22
Epoch cursor	2964, the current epoch
Unstake wait period	7 days
Stake target, APR min, max, step	30 percent, 2.5, 75, 1
Proposal threshold	0.1 percent of shares
Claims managers	none ever
Primary voting app	support 50 percent, quorum 50 percent, 17 votes, none open
Secondary voting app	support 50 percent, quorum 15 percent, 135 votes, 133 and 134 open
Secondary vote 133	created 2026-10-27 18:18:47 UTC by `0x8566…6671`, snapshot block 26,285,794, closes 2026-11-03 18:18:47 UTC, yea 259,876.41 shares, nay 0, of 15,701,999.75
Secondary vote 134	created 2026-10-27 18:51:59 UTC by `0xe062…6291`, snapshot block 26,285,960, closes 2026-11-03 18:51:59 UTC, yea 259,876.41 shares, nay 0, of 15,701,999.75
Registry executor 1	`0x5cEb19e1890f677c3676d5ecDF7c501eBA01A054`, enabled, no other
Kernel recovery vault	primary Agent
Primary Agent	6,083,873.527701856751553400 API3, 0 ETH
Secondary Agent	59,419.713966 USDC, 1,547,987 PNK, 249,999.99 wPOKT, 0.057118793443618210 ETH
Vault Safe	2,940,786.09 API3, 3,371,901 pool shares by delegation
Treasury Safe	0 API3, not a burner
TimelockManager `0xFaef86994a37F1c8b2A5c73648F07dd4eFF02baA`	owned by `0xf26304D6Cd67DFA199b17c9E850149bF77285611`, not the DAO

Safes

Owners of all three: `0x09dEa15D4E1F82787e181F23788Dc3f4b3d3DAE4`, `0xA3647CfF715457e34Eb6A0c7C0fD24BD2B7f53Ca`, `0x642379Ed64c6b65d8164075f2565BAb6825d1214`, `0xe0625a7B539090130f6eDCdef208f6be129a6291`, `0xBF0e96198628382A4644A985eA63AE19d53264D9`. Threshold 3 on each. Singletons `0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552` (1.3.0), `0x41675C099F32341bf84BFc5382aF534df5C7461a` (1.4.1),

`0xFf51A5898e281Db6DfC7855790607438dF2ca44b` (1.5.0), fallback handlers `0xf48f2B2d2a534e402487b3ee7C18c33Aec0Fe5e4`, `0xfd0732Dc9E303f09fCEf3a7388Ad10A83459Ec99`, `0x3EfCBb83A4A7AfcB4F68D501E2c2203a38be77f4`. No modules, no guard.

Appendix B: the audited revision

Commit `a920489e0e0b0eab1abb2f9858623f3c1ffe419e` on `main`, the merge of pull request 12, one change on top of `0e10333`. SHA-256 of `src/MigrationScript.sol`: `c27742141b81dd1926ac5bd8f94a2773e48e42cced8bb6a69c9647e4e6ec65d3`. The first 16 hex characters of the SHA-256 of every tracked file:

```
04bac673936707c2  .github/workflows/ci.yml
ec195d4af2697a9e  .gitignore
c50004a2be2a67ce  README.md
2c8be0dca5c335c2  foundry.toml
4f564afc33665242  proposal/description.md
240a3e0d37d2e86b  proposal/generated/.gitignore
54dc06397cfd80e1  proposal/title.txt
be7aafa97cdb486e  script/CreateVote.s.sol
095363a8c2942a5c  script/VerifyVote.s.sol
eec1bf2ab1e4197c  scripts/decode-script.mjs
b5a98cb1bdaf0add  scripts/pick-voters.sh
0661a45c935c8564  scripts/update-readme.mjs
6bc970d43ab4084d  setup.sh
a53d9f3c1f94026b  src/Interfaces.sol
c27742141b81dd19  src/MigrationScript.sol
ce8b7a56d5e08e02  test.sh
ef0bed481980c778  test/Base.t.sol
1d5b620fa3dccf3a  test/MigrationProposal.t.sol
3d47940c08fadb78  test/dao/aragon.t.sol
f3a5d34ecc80c637  test/dao/aragon.tree
15758481f537de39  test/data/voters.json
b2a60eb3020808a3  test/pool/rewards.t.sol
21c3532bf7b156f5  test/pool/rewards.tree
5b255e66c56cb36a  test/pool/staking.t.sol
a58bc3dffff18976  test/pool/staking.tree
b0faa83e2903ca0c  test/vote/execute.t.sol
a65e276bae1999af  test/vote/execute.tree
1091434f320b0512  test/vote/script.t.sol
ca0f566fb91a5c58  test/vote/script.tree
```

Nine files changed from version 0.1: the script, the README, `CreateVote.s.sol`, the fixture, `test/MigrationProposal.t.sol` and the two vote trees with their tests. Version 1.1 audits `4740c29551fc8be8bd21ecab9d5baeec8d8460d2`, the merge of pull request 15 on top of it. It changes two files: `script/VerifyVote.s.sol` to `ec01cdd8c5335c05` and `README.md` to `5c74fdf61015eba4`. The script that builds the bytes is untouched and the bytes keep the hash of section 2. The IPFS link that goes into `proposal/description.md` before posting changes that file's hash and the metadata, not the bytes.

Toolchain: Foundry 1.8.5, solc 0.8.20, forge-std v1.17.0, bulloak 0.8.1, Node 24.

Appendix C: redoing this review

The repository's own checks:

```
./setup.sh
./test.sh                       # the suite, 28 tests at the latest block
forge script script/CreateVote.s.sol \
  --rpc-url <archive> --fork-block-number 26287968
node scripts/decode-script.mjs proposal/generated/script.hex
forge script script/VerifyVote.s.sol \
  --sig "run(uint256,uint256)" <id> <build block>
```

The review's own checks, described so that they can be redone with `cast` and any archive RPC:

- **Independent decoding.** Walk `proposal/generated/script.hex`: a 4 byte spec id `0x00000001`, then entries of a 20 byte target, a 4 byte calldata length and the calldata. An entry whose target is the primary Agent and whose selector is `execute(address,uint256,bytes)` is unwrapped to its inner target, value and calldata. Decode selectors against the ten signatures the vote uses (`grantPermission`, `revokePermission`, `removePermissionManager`, `transfer`, `execute`, `updateMinterStatus`, `mint`, `transferOwnership`, `changeSupportRequiredPct`, `changeMinAcceptQuorumPct`) and role hashes against the seven role names. Compare caller, target, function, addresses, roles and amounts with the README's table row by row.
- **Permissions.** `eth_getLogs` on the ACL address from block 12,786,000 to the latest, all topics. Decode `SetPermission(address,address,bytes32,bool)`, `SetPermissionParams(address,address,bytes32,bytes32)` and `ChangePermissionManager(address,bytes32,address)`, keep the last event per key. The same on the Kernel (`NewAppProxy`, `SetApp`) and the registry (`EnableExecutor`).
- **Ownership and roles.** `eth_getLogs` with no address and topics `[OwnershipTransferred(address, address), null, <agent>]` and `[RoleGranted(bytes32,address,address), null, <agent>]` for both Agents, from block 12,786,000 to the latest, in chunks the RPC accepts.
- **Holdings.** `eth_getLogs` with no address and topics `[Transfer(address,address,uint256), null, <agent>]` (ERC20 with three topics, ERC721 with four), `[TransferSingle(...), null, null, <agent>]` and `[TransferBatch(...), null, null, <agent>]`. Then `balanceOf` at the block for every contract seen.
- **Parameters and Safes.** The `cast call` reads listed in Appendix A, plus `getOwners`, `getThreshold`, `getModulesPaginated(0x1, 10)`, the guard slot `0x4a204f62…34c8`, the fallback handler slot `0x6c9a6c4a…18d5` and storage slot 0 for the singleton on each Safe, compared with `safe-global/safe-deployments` for the version each reports.
- **The secondary votes.** `cast call <secondary voting app> 'getVote(uint256)(bool,bool,uint64,uint64,uint64,uint64,uint256,uint256,uint256,bytes)' 133` and `134`, the last value being the script. Rebuild it as the 4 byte spec id `0x00000001`, the secondary Agent's address, a 4 byte length and the calldata `execute(token, 0, transfer(agentSafe, balance))` with the token's `balanceOf(secondaryAgent)` at the block, and compare the keccak256. The creator is the second indexed topic of the `StartVote` log in the block after the snapshot block.
- **Rehearsals.** Foundry tests on a mainnet fork on top of `test/Base.t.sol`, as described under each finding, with `vm.prank` for the third parties and the Vault's votes, `vm.warp` past 2026-11-03 18:52 UTC for the secondaries' close, and `gasleft()` around `newVote` and `executeVote` for the gas.

Appendix D: gas

Measured on a mainnet fork at the latest block on 2026-10-28, after block 26,287,968, from an externally owned account, so without the Safe's own overhead on the posting. Version 0.1 measured 3,361,608 and 1,942,684 for the 28 call script.

Transaction	Gas
`newVote` with the 26 call script and the metadata	2,995,478
`executeVote`, the 26 calls	1,549,869
The same with the pool hardening call of F-3	3,181,913 and 1,680,063